ASSIGNMENT-10.5

HT.NO:2303A51731

Task Description #1 – Variable Naming Issues

Task: Use AI to improve unclear variable names.

Sample Input Code:

def f(a, b):

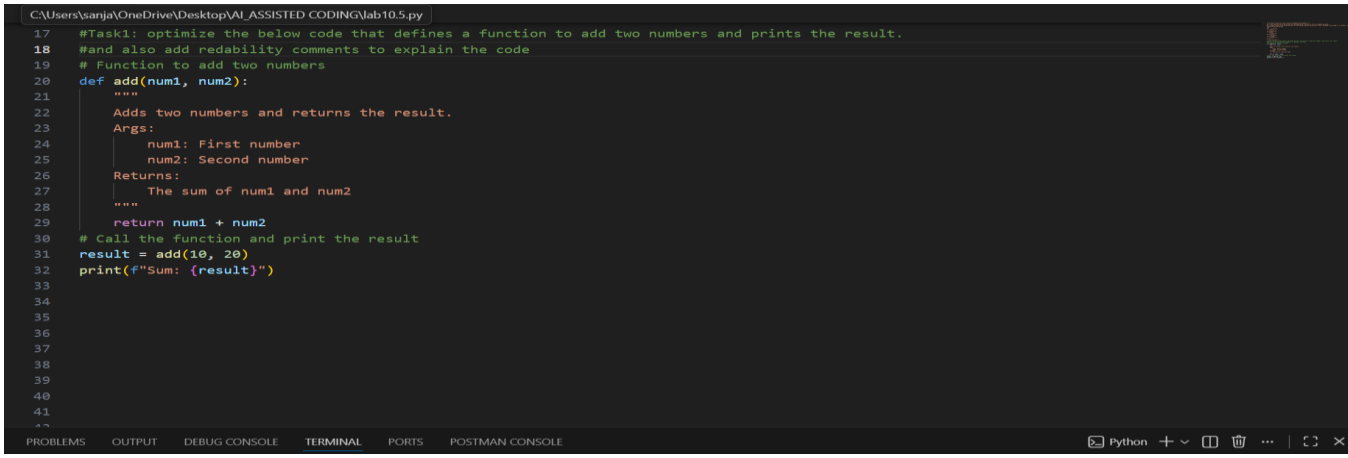
return a + b

print(f(10, 20))

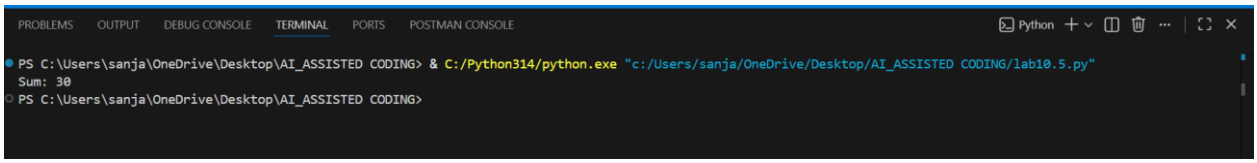
Expected Output:

• Code rewritten with meaningful function and variable names.

CODE:



OUTPUT:



Task Description #2 – Missing Error Handling

Task: Use AI to add proper error handling.

Sample Input Code:

def divide(a, b):

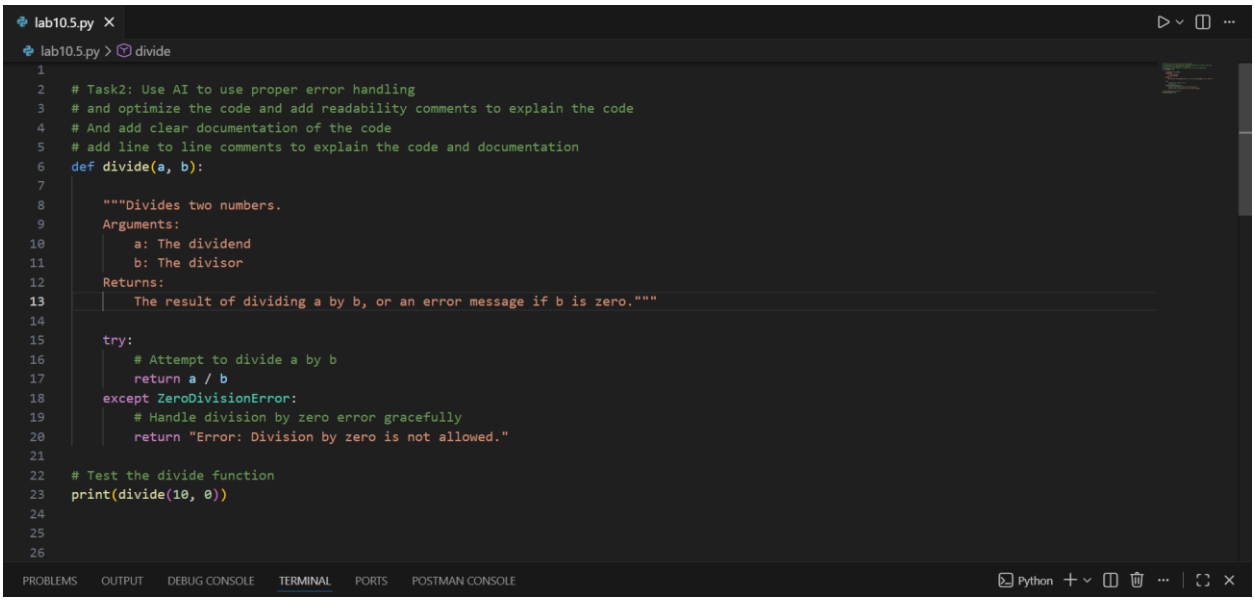
return a / b

print(divide(10, 0))

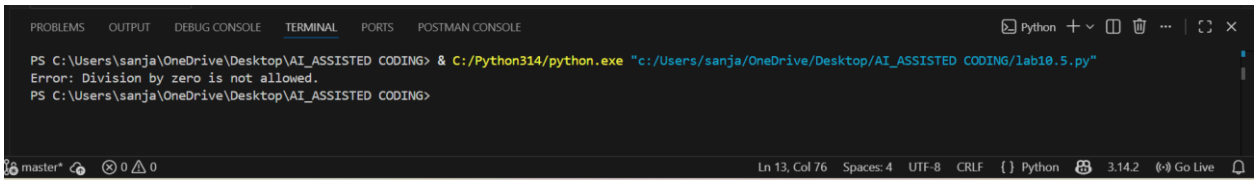
Expected Output:

• Code with exception handling and clear error messages

CODE:



OUTPUT:



Task Description #3: Student Marks Processing System

The following program calculates total, average, and grade of a student, but it has poor readability, style issues, and no error handling.

```
marks=[78,85,90,66,88]
t=0
for i in marks:
t=t+i
a=t/len(marks)
if a>=90:
print("A")
elif a>=75:
print("B")
elif a>=60:
print("C")
else:
print("F")
```

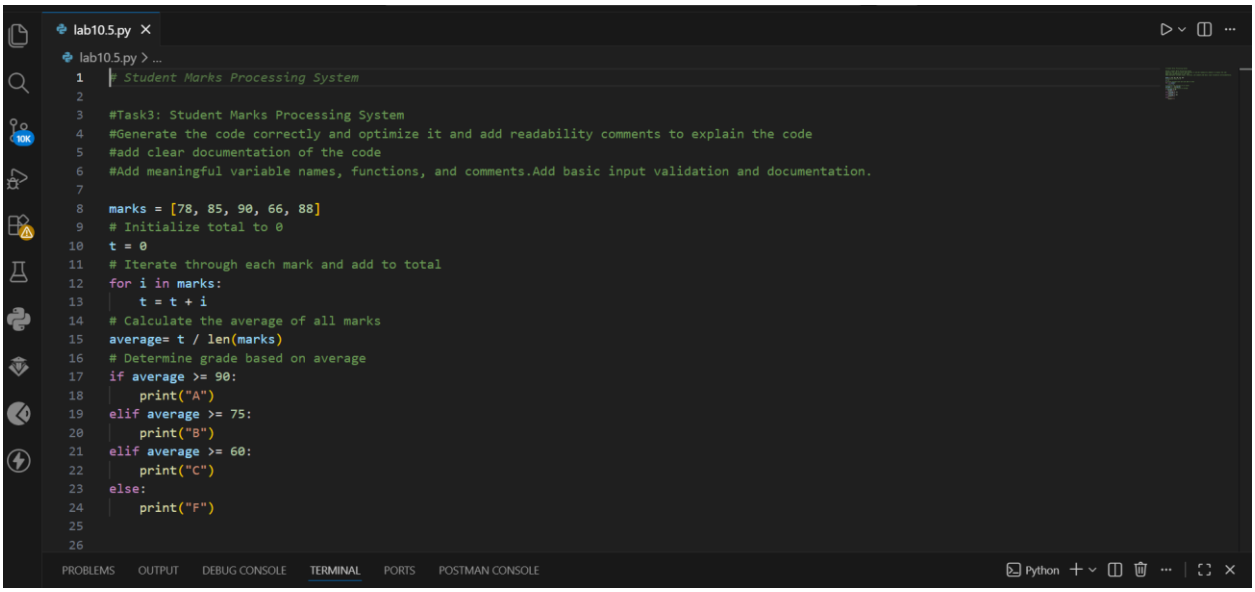
Task:

• Use AI to refactor the code to follow PEP 8 standards.

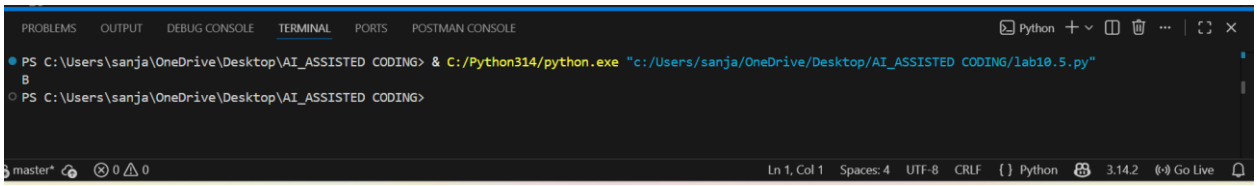
• Add meaningful variable names, functions, and comments.

• Add basic input validation and documentation.

CODE:



OUTPUT:



Explanation of Refactoring Changes:

1. **Meaningful Names (PEP 8)**:
   - `marks` -> `student_marks` : More descriptive of the data.
   - `t` -> `total_marks` : Clearly indicates what the variable holds.
   - `i` -> `mark` : Better name for individual items in the loop.
   - `a` -> `average_mark` : Explicitly states the calculated value.
   - `f, a, b, c` -> `calculate_student_grade` : A function name that explains its purpose.
2. **Function Encapsulation**:
   - The logic is now contained within a function `calculate_student_grade`. This improves modularity, reusability, and testability. It also makes the code easier to understand by breaking it into logical units.
3. **Comments and Docstrings (PEP 257 & PEP 8)**:
   - A docstring has been added to the `calculate_student_grade` function, explaining its purpose, arguments, and return value. This is crucial for documentation and helps anyone reading the code understand its functionality without needing to delve into the implementation details.
   - Inline comments are used to clarify specific steps, such as calculating total marks or determining the grade.
4. **PEP 8 Styling**:
   - Consistent indentation (4 spaces) and spacing around operators (`=`, `+`, `/`) are applied.
   - Blank lines are used to separate logical blocks within the function for improved readability.
   - Function and variable names follow `snake_case` conventions.

5. **Basic Input Validation and Error Handling**:
   - **Empty List Check**: Added `if not student_marks:` to handle cases where an empty list is passed, preventing `ZeroDivisionError` and providing a user-friendly message.
   - **Invalid Mark Check**: Added a loop to ensure that each `mark` in `student_marks` is a number and falls within a valid range (0-100). This makes the function more robust against unexpected input.
6. **Clarity in Output**:
   - The `print` statements now provide more context, indicating what is being printed (e.g., "Total Marks:", "Average Mark:", "Final Grade:").

These changes significantly improve the code's maintainability, readability, and robustness, making it easier for others (or your future self) to understand and modify.

Task Description #4: Use AI to add docstrings and inline comments

to the following function.

def factorial(n):

result = 1

for i in range(1,n+1):

result *= i

return result

CODE:

lab10.5.py

```python
#Task4: add docstrings and inline comments to the following function.
#Add clear documentation line to line comments to explain the code and optimize it.
def factorial(num):
    """Calculates the factorial of a number.
    Arguments:
        num: The number to calculate factorial for (must be non-negative integer)
    Returns:
        The factorial of num, or an error message if input is invalid"""
    # Validate that input is a non-negative integer
    if not isinstance(num, int) or num < 0:
        return "Error: Input must be a non-negative integer."
    # Handle base case: factorial of 0 is 1
    if num == 0:
        return 1
    # Calculate factorial by multiplying all integers from 1 to num
    result = 1
    for i in range(1, num + 1):
        result *= i
    return result
print(factorial(4))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python

OUTPUT:

Task Description #5: Password Validation System (Enhanced)

The following Python program validates a password using only a minimum length check, which is insufficient for real-world security requirements.

```
pwd = input("Enter password: ")
if len(pwd) >= 8:
print("Strong")
else:
print("Weak")
```

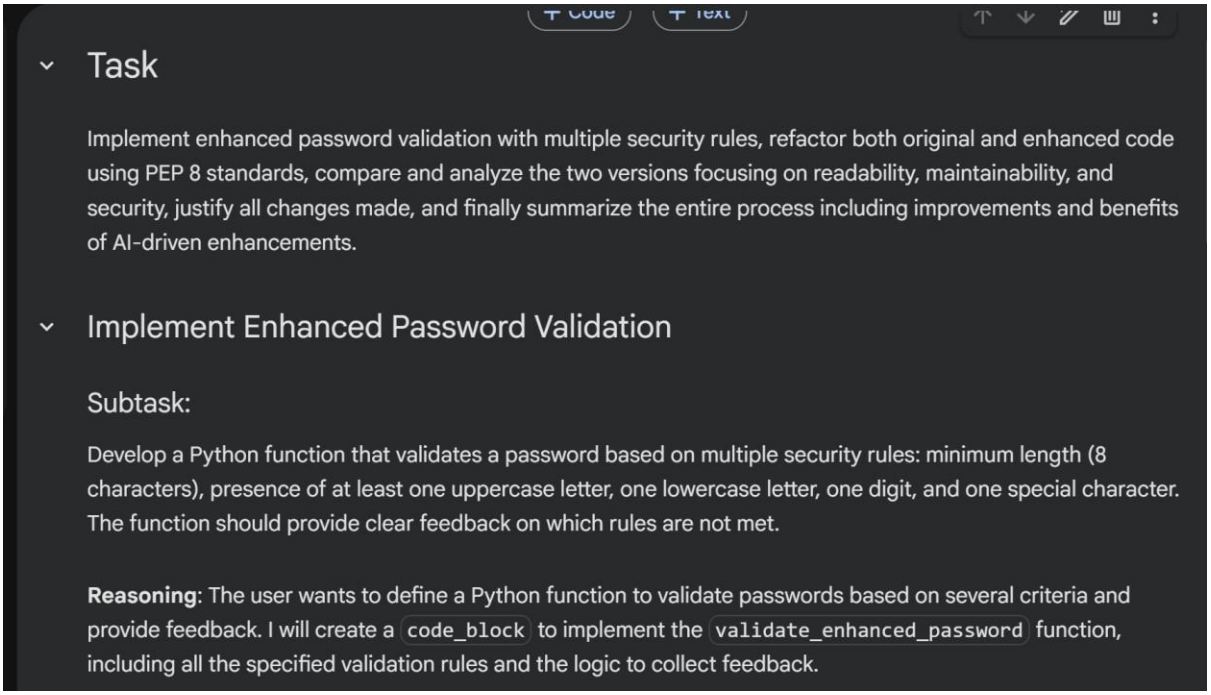
Task:

1. Enhance password validation using AI assistance to include multiple security rules such as:
   - o Minimum length requirement
   - o Presence of at least one uppercase letter
   - o Presence of at least one lowercase letter
   - o Presence of at least one digit
   - o Presence of at least one special character
2. Refactor the program to:
   - o Use meaningful variable and function names
   - o Follow PEP 8 coding standards
   - o Include inline comments and a docstring
3. Analyze the improvements by comparing the original and AI-enhanced versions in terms of:
   - o Code readability and structure
   - o Maintainability and reusability

o Security strength and robustness

4. Justify the AI-generated changes, explaining why each added rule and refactoring decision improves the overall quality of the program.

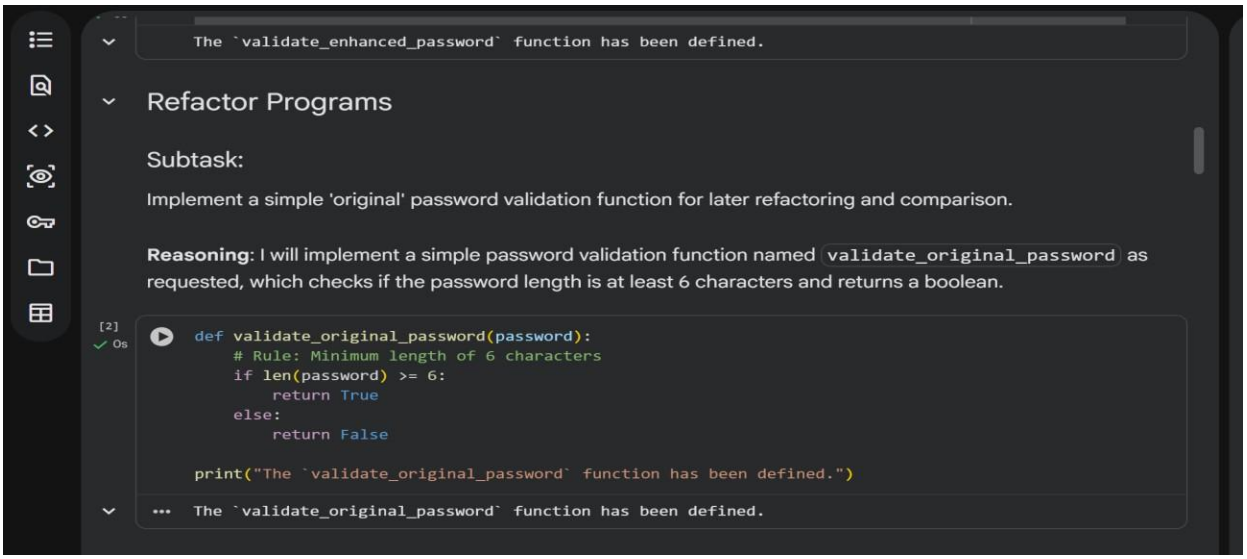
TASK EXPLANATION:



CODE:

lab10.5.py

```python
#Task5: Password Validation System (Enhanced)
#Generate the code correctly and optimize it and add readability comments to explain the code
#Add clear documentation of the code
#Add meaningful variable names, functions, and comments.Add basic input validation and documentation.

pwd = input("Enter password: ")
if len(pwd) >= 8:
 print("Strong")
else:
 print("Weak")
import re
def is_strong_password(password):
    """
    Check if the provided password meets the security criteria:
    - Minimum length of 8 characters
    - Contains at least one uppercase letter
    - Contains at least one lowercase letter
    - Contains at least one digit
    - Contains at least one special character
    """
    if len(password) < 8:
        return False
    if not re.search(r'[A-Z]', password):
        return False
    if not re.search(r'[a-z]', password):
        return False
    if not re.search(r'[0-9]', password):
        return False
    if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
        return False
    return True
def main():
    """Main function to execute the password validation process."""
    password = input("Enter password: ")
    if is_strong_password(password):
        print("Strong")
    else:
        print("Weak")
if __name__ == "__main__":
    main()
```



```
The `validate_enhanced_password` function has been defined.
```

Refactor Programs

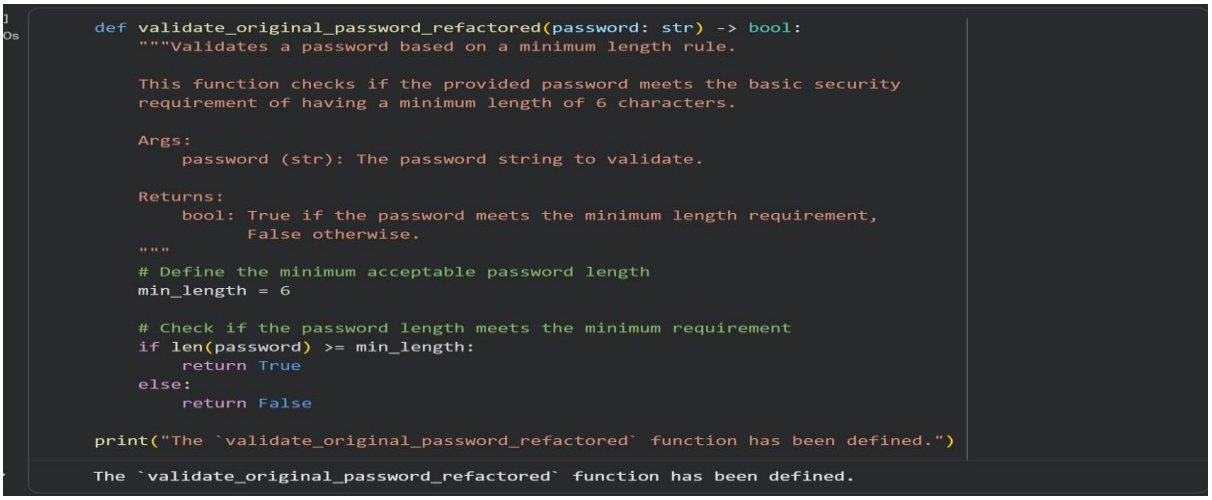
Subtask:

Implement a simple 'original' password validation function for later refactoring and comparison.

Reasoning: I will implement a simple password validation function named `validate_original_password` as requested, which checks if the password length is at least 6 characters and returns a boolean.

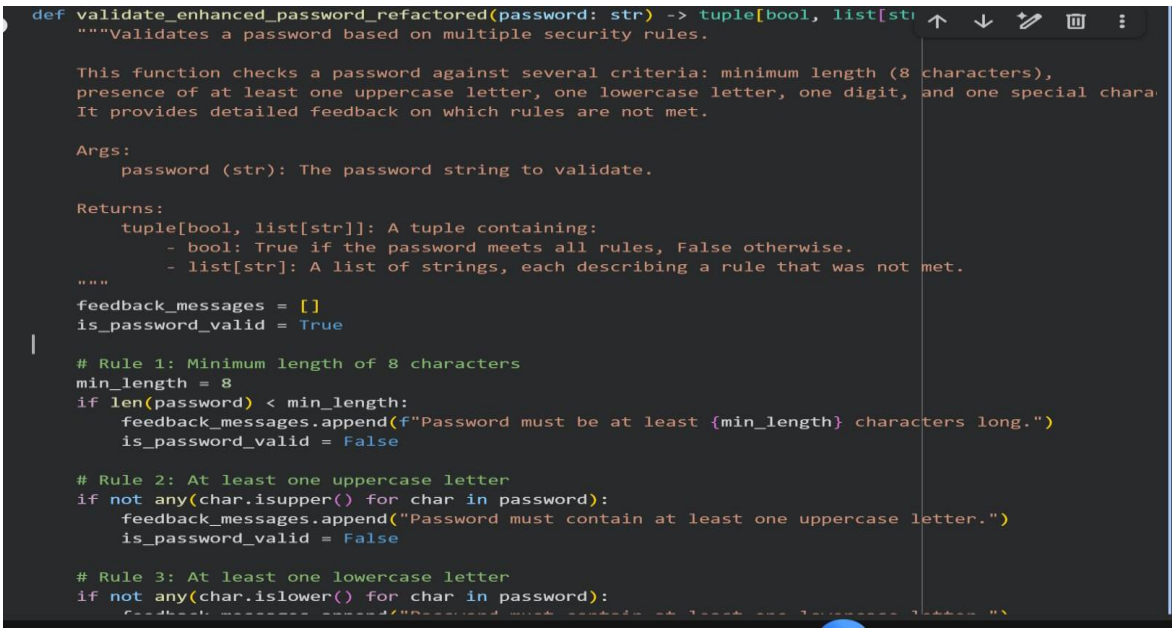
```
[2]
✓ 0s
def validate_original_password(password):
    # Rule: Minimum length of 6 characters
    if len(password) >= 6:
        return True
    else:
        return False

print("The `validate_original_password` function has been defined.")
```

```
... The `validate_original_password` function has been defined.
```

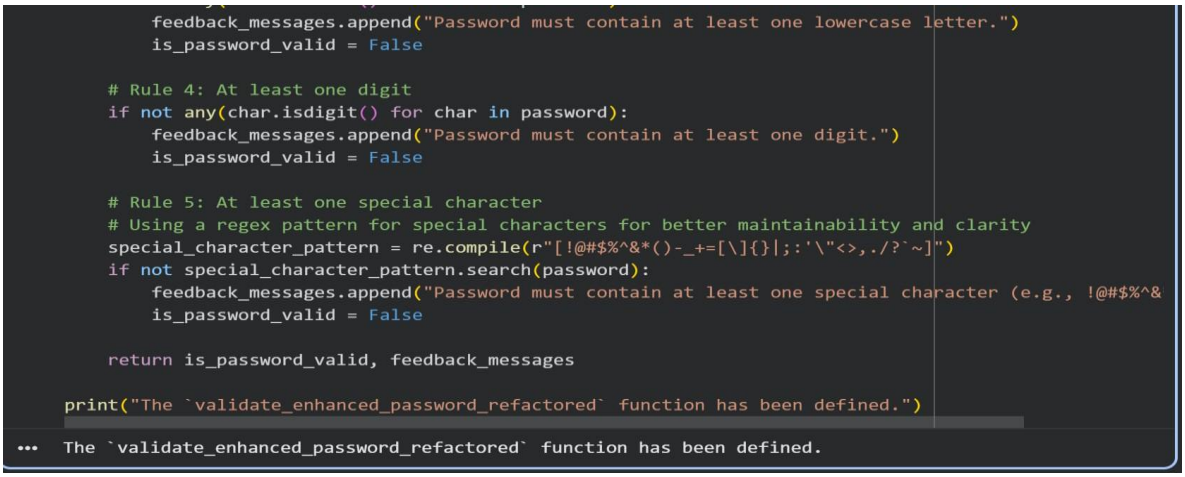


```
0s
def validate_original_password_refactored(password: str) -> bool:
    """Validates a password based on a minimum length rule.

    This function checks if the provided password meets the basic security
    requirement of having a minimum length of 6 characters.

    Args:
        password (str): The password string to validate.

    Returns:
        bool: True if the password meets the minimum length requirement,
              False otherwise.
    """
    # Define the minimum acceptable password length
    min_length = 6

    # Check if the password length meets the minimum requirement
    if len(password) >= min_length:
        return True
    else:
        return False

print("The `validate_original_password_refactored` function has been defined.")
```

```
The `validate_original_password_refactored` function has been defined.
```



```
def validate_enhanced_password_refactored(password: str) -> tuple[bool, list[str
    """Validates a password based on multiple security rules.

    This function checks a password against several criteria: minimum length (8 characters),
    presence of at least one uppercase letter, one lowercase letter, one digit, and one special chara
    It provides detailed feedback on which rules are not met.

    Args:
        password (str): The password string to validate.

    Returns:
        tuple[bool, list[str]]: A tuple containing:
            - bool: True if the password meets all rules, False otherwise.
            - list[str]: A list of strings, each describing a rule that was not met.
    """
    feedback_messages = []
    is_password_valid = True

    # Rule 1: Minimum length of 8 characters
    min_length = 8
    if len(password) < min_length:
        feedback_messages.append(f"Password must be at least {min_length} characters long.")
        is_password_valid = False

    # Rule 2: At least one uppercase letter
    if not any(char.isupper() for char in password):
        feedback_messages.append("Password must contain at least one uppercase letter.")
        is_password_valid = False

    # Rule 3: At least one lowercase letter
    if not any(char.islower() for char in password):
```



```
            feedback_messages.append("Password must contain at least one lowercase letter.")
            is_password_valid = False

        # Rule 4: At least one digit
        if not any(char.isdigit() for char in password):
            feedback_messages.append("Password must contain at least one digit.")
            is_password_valid = False

        # Rule 5: At least one special character
        # Using a regex pattern for special characters for better maintainability and clarity
        special_character_pattern = re.compile(r"[!@#$%^&*()-_+=[\]{}|;:'\"<>,./?`~]")
        if not special_character_pattern.search(password):
            feedback_messages.append("Password must contain at least one special character (e.g., !@#$%^&
            is_password_valid = False

        return is_password_valid, feedback_messages

    print("The `validate_enhanced_password_refactored` function has been defined.")
```

```
...  The `validate_enhanced_password_refactored` function has been defined.
```

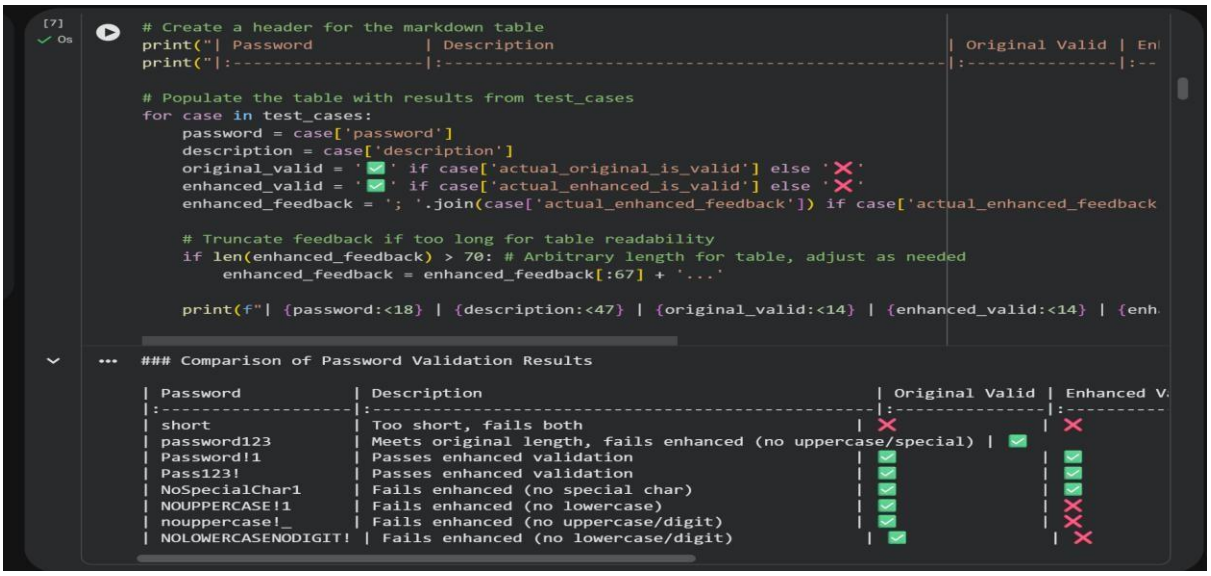
TEST CASES:

```
test_cases = [
  {
    "password": "short",
    "description": "Too short, fails both",
    "expected_original": False,
    "expected_enhanced_is_valid": False,
    "expected_enhanced_feedback_contains": [
      "Password must be at least 8 characters long.",
      "Password must contain at least one uppercase letter.",
      "Password must contain at least one digit.",
      "Password must contain at least one special character."
    ]
  },
  {
    "password": "password123",
    "description": "Meets original length, fails enhanced (no uppercase/special)",
    "expected_original": True,
    "expected_enhanced_is_valid": False,
    "expected_enhanced_feedback_contains": [
      "Password must contain at least one uppercase letter.",
      "Password must contain at least one special character."
    ]
```

```
},
{
  "password": "Password!1",
  "description": "Passes enhanced validation",
  "expected_original": True,
  "expected_enhanced_is_valid": True,
  "expected_enhanced_feedback_contains": []
},
{
  "password": "Pass123!",
  "description": "Passes enhanced validation",
  "expected_original": True,
  "expected_enhanced_is_valid": True,
  "expected_enhanced_feedback_contains": []
},
{
  "password": "NoSpecialChar1",
  "description": "Fails enhanced (no special char)",
  "expected_original": True,
  "expected_enhanced_is_valid": False,
  "expected_enhanced_feedback_contains": [
    "Password must contain at least one special character."
  ]
},
{
  "password": "NOUPPERCASE!1",
  "description": "Fails enhanced (no lowercase)",
  "expected_original": True,
  "expected_enhanced_is_valid": False,
  "expected_enhanced_feedback_contains": [
    "Password must contain at least one lowercase letter."
```

```
        ]
    },
    {
        "password": "nouppercase!_",
        "description": "Fails enhanced (no uppercase/digit)",
        "expected_original": True,
        "expected_enhanced_is_valid": False,
        "expected_enhanced_feedback_contains": [
            "Password must contain at least one uppercase letter.",
            "Password must contain at least one digit."
        ]
    },
    {
        "password": "NOLOWERCASENODIGIT!",
        "description": "Fails enhanced (no lowercase/digit)",
        "expected_original": True,
        "expected_enhanced_is_valid": False,
        "expected_enhanced_feedback_contains": [
            "Password must contain at least one lowercase letter.",
            "Password must contain at least one digit."
        ]
    }
]
print(f"Defined {len(test_cases)} password test cases.")
```

OUTPUT:



```
[7]
✓ 0s
# Create a header for the markdown table
print("| Password           | Description                                                | Original Valid | En
print("|:-------------------|:-----------------------------------------------------------|:---------------|:--

# Populate the table with results from test_cases
for case in test_cases:
    password = case['password']
    description = case['description']
    original_valid = '✅' if case['actual_original_is_valid'] else '❌'
    enhanced_valid = '✅' if case['actual_enhanced_is_valid'] else '❌'
    enhanced_feedback = '; '.join(case['actual_enhanced_feedback']) if case['actual_enhanced_feedback

    # Truncate feedback if too long for table readability
    if len(enhanced_feedback) > 70: # Arbitrary length for table, adjust as needed
        enhanced_feedback = enhanced_feedback[:67] + '...'

    print(f"| {password:<18} | {description:<47} | {original_valid:<14} | {enhanced_valid:<14} | {enh
```

```
### Comparison of Password Validation Results

| Password           | Description                                                | Original Valid | Enhanced V
|:-------------------|:-----------------------------------------------------------|:---------------|:----------
| short              | Too short, fails both                                      | ❌             | ❌
| password123        | Meets original length, fails enhanced (no uppercase/special) | ✅          |
| Password!1         | Passes enhanced validation                                 | ✅             | ✅
| Pass123!           | Passes enhanced validation                                 | ✅             | ✅
| NoSpecialChar1     | Fails enhanced (no special char)                           | ✅             | ✅
| NOUPPERCASE!1      | Fails enhanced (no lowercase)                              | ✅             | ❌
| nouppercase!_      | Fails enhanced (no uppercase/digit)                        | ✅             | ❌
| NOLOWERCASENODIGIT! | Fails enhanced (no lowercase/digit)                       | ✅             | ❌
```